A *destination binding* can be either an input channel or an output channel. By default, Spring Cloud Stream maps each binding (both input and output) to an exchange in RabbitMQ (a *topic exchange*, to be more precise). Furthermore, for each input binding, it binds a queue to the related exchange. That's the queue from which consumers receive and process events. This setup provides all the plumbing for implementing event-driven architectures based on the pub/sub model.

In the following sections, I'll tell you more about destination bindings in Spring Cloud Stream and how they relate to exchanges and queues in RabbitMQ.

## *UNDERSTANDING DESTINATION BINDINGS*

As you can see in figure 10.10, destination bindings are an abstraction representing a bridge between application and broker. When using the functional programming model, Spring Cloud Stream generates an input binding for each function accepting input data, and an output binding for each function returning output data. Each binding is assigned a logical name following this convention:

- Input binding: `<functionName>` + `-in-` + `<index>`
- Output binding: `<functionName>` + `-out-` + `<index>`

Unless you use partitions (for example, with Kafka), the `<index>` part of the name will always be `0`. The `<functionName>` is computed from the value of the `spring.cloud.function.definition` property. In case of a single function, there is a one-to-one mapping. For example, if in Dispatcher Service we only had one function called